

2026-05-18-round37-memory-persistence-real-backends-design

Round 37 — Memory-Persistence Real-Backend Wiring — Design

Date: 2026-05-18 **Status:** Approved (operator blanket-approved all sections) **Programme:** HelixCode anti-bluff campaign, round 37 (follow-on from round 31 partial fix + round 35 RedisMemoryProvider audit finding) **Constitutional anchors:** CONST-035, CONST-042 (no-secret-leak), CONST-050(A) + CONST-050(B), CONST-051(B), CONST-060, CONST-061, Article XI §11.9

1. Architecture

Premise: Round 31's fix made `Health()` honest (real `Ping()` + sentinel for nil-client) but left the data path fake — `RedisMemoryProvider` is an in-memory map masquerading as Redis; same for `Memcached`. `Storage.SyncRecording` marks recordings “synced” with no S3 upload. `VectorDB.pgvector.Search/Get` return sentinels. End-user data is being lost across restarts despite the API contract promising persistence.

Round 37 closes the gap by giving each provider a **real client field acquired at construction from configuration**. The sentinel error becomes a strict precondition: if `client == nil` at method entry, refuse loudly; otherwise delegate to the real client.

Three submodules touched, all decoupled per CONST-051(B): 1. `helix_code/internal/memory/` — `RedisMemoryProvider` + `MemcachedMemoryProvider` get real clients 2. `submodules/storage/` — `SyncRecording` wires real S3 `PutObject` 3. `submodules/vector_db/` — `pgvector Search + Get` use real `pgxpool`

Config flow: Each provider's constructor takes a typed config struct sourced from the parent application's `.env` (CONST-042). Absent/invalid config → constructor returns the sentinel; provider is created in a “stub” state where every data method returns the sentinel. Preserves the round-31 invariant that an unconfigured provider is safe to hold and surfaces the gap loudly on first use.

Non-goals: - Connection pooling tuning (use sensible defaults; perf round later) - Failover/replica strategies (single-endpoint sufficient) - Runtime backend swap (operator restarts on config change)

2. Components

2.1 helix_code/internal/memory/memory_manager.go

RedisMemoryProvider struct gains: - client *redis.Client field (from github.com/redis/go-redis/v9 — already in go.mod, verify; if not, go-redis/v9 is canonical) - config RedisConfig (URL, Password, DB, TLS bool, DialTimeout)

Constructor: NewRedisMemoryProvider(cfg RedisConfig)
*RedisMemoryProvider — if cfg.URL == "", returns a provider with client = nil (data methods return ErrRedisClientNotInitialized). Otherwise calls redis.NewClient(opts) + invokes Ping(ctx) once at construction; on failure stores the client but Ping error surfaces on next data call.

Data methods (Get, Set, Delete, List, Health, Close): - Each guards if p.client == nil { return ..., ErrRedisClientNotInitialized } - Get → p.client.Get(ctx, key).Bytes(); map redis.Nil to (nil, ErrKeyNotFound) (existing sentinel) - Set → p.client.Set(ctx, key, value, ttl).Err() - Delete → p.client.Del(ctx, keys...).Err() - List (prefix scan) → p.client.Scan(ctx, 0, prefix+"*", 100).Iterator() paginated - Health → existing real Ping() (round-31; no change) - Close → p.client.Close()

MemcachedMemoryProvider struct gains: - client *memcache.Client field (from github.com/bradfitz/gomemcache/memcache) - config MemcachedConfig (Servers []string, Timeout, MaxIdleConns)

Constructor: same pattern — empty Servers → nil-client mode → sentinels. Otherwise memcache.New(cfg.Servers...).

Data methods: - Get → p.client.Get(key) → ErrCacheMiss mapped to ErrKeyNotFound - Set → p.client.Set(&memcache.Item{Key, Value, Expiration}) - Delete → p.client.Delete(key) → ErrCacheMiss ignored (idempotent) - List → **not supported by memcached protocol**; return ErrListNotSupported sentinel + doc-comment explaining memcached has no scan API - Health → existing real Ping (round-31) - Close → p.client.Close() (or no-op if not supported)

2.2 submodules/storage/ SyncRecording

Current state (per round 21 close-out): SyncRecording returns ErrS3UploadNotWired sentinel.

Round 37 wires it: existing s3Client field (or add one) of type *s3.Client (AWS SDK v2: github.com/aws/aws-sdk-go-v2/service/s3).

SyncRecording(ctx, session) **body:** 1. If s.s3Client == nil: return ErrS3UploadNotWired (unchanged round-21 behaviour) 2. Read recording from local disk: data, err := os.ReadFile(session.LocalPath) 3. Detect size: if > 5 MB, use multipart upload via manager.NewUploader(s.s3Client).Upload(...); else single PutObject 4. On success: session.S3Key = key, session.Status = RecordingStatusSynced, session.SyncedAt = time.Now() 5. On failure: session.Status = RecordingStatusFailed, wrap and return error

Constructor NewSyncManager(cfg StorageConfig): - If cfg.Bucket == "" || cfg.Region == "" → s3Client = nil → all data methods return ErrS3UploadNotWired - Else aws.NewConfig(...) from env-sourced credentials (CONST-042; never hardcoded) → s3.NewFromConfig(cfg)

CloudFront RSA-SHA1 signing (round-21 sibling fix ErrCloudFrontSigningNotWired) — **out of scope for round 37**; deferred to round 38 candidate.

2.3 submodules/vector_db/ pgvector

Current state (per round 22 close-out): Search + Get return ErrPgvectorSearchNotWired / ErrPgvectorGetNotWired.

Round 37 wires both: *pgxpool.Pool field on the pgvector client struct, from github.com/jackc/pgx/v5/pgxpool.

Search(ctx, query, k) **body:** 1. If c.pool == nil: return ErrPgvectorSearchNotWired (unchanged) 2. Build query: SELECT id, content, embedding <=> \$1 AS distance FROM vectors ORDER BY distance LIMIT \$2 (uses pgvector <=> cosine-distance operator) 3. rows, err := c.pool.Query(ctx, sql, query, k) 4. Scan into []SearchResult; return

Get(ctx, id) **body:** - c.pool.QueryRow(ctx, "SELECT content FROM vectors WHERE id = \$1", id).Scan(&content) - pgx.ErrNoRows → ErrVectorNotFound (introduce if absent)

Constructor NewPgvectorClient(cfg PgvectorConfig): - If cfg.DSN == "" → pool = nil → sentinels - Else pgxpool.New(ctx, cfg.DSN) + register pgvector.RegisterTypes(ctx, conn) once per acquired conn

3. Data flow

End user → application/Service → MemoryProvider.Set("key", value, ttl)
↓
[Round 36: returns nil silently to in-memory
Round 37: client.Set(ctx, key, value, ttl).
↓
Real Redis container
↓
Data survives restart ✓

Same shape for Storage S3 upload (recording bytes → S3 PutObject → object actually exists in bucket) and pgvector (query embedding → cosine distance → real ANN result set).

4. Error handling

Sentinel taxonomy (per provider, all already named):

Provider	Nil-client sentinel	Operation-failure pattern
Redis	ErrRedisClientNotInitialized	wrap <code>client.X.Err()</code> with <code>fmt.Errorf("redis %s: %w", op, err)</code>
Memcached	ErrMemcachedClientNotInitialized	wrap; ignore <code>ErrCacheMiss</code> for Delete
S3	ErrS3UploadNotWired (nil-client) + new <code>ErrS3UploadFailed</code> wrapping AWS error	distinguish nil-client (config gap) from upload failure (network/perms)
pgvector	<code>ErrPgvectorSearchNotWired</code> + <code>ErrPgvectorGetNotWired</code> (existing); new <code>ErrVectorNotFound</code> for missing row	wrap pgx errors

Round-31 invariant preserved: a caller holding an unconfigured provider can safely call `Health()` — it returns the sentinel without panicking. Round 37 extends this guarantee to every data method.

No silent fallbacks. If real client returns an error, it bubbles to the caller with full context (operation + key/bucket/path + wrapped underlying error). Never log-and-return-nil.

5. Testing

Per operator's chosen test approach (reuse existing `helix_code` docker-compose + SKIP-OK for submodules):

5.1 `helix_code/internal/memory/` — **real-backend integration tests via docker-compose**

Extend `docker-compose.full-test.yml` if Redis/Memcached aren't already there (they likely are per CLAUDE.md §3.4 since this is a multi-backend platform; subagent verifies + extends if missing).

Tests: - `TestRedisMemoryProvider_Set_Get_Roundtrip` — Set then Get returns same value - `TestRedisMemoryProvider_Get_NotFound` — returns `ErrKeyNotFound` for unknown key - `TestRedisMemoryProvider_Set_TTL_Expires` — sets short TTL, sleeps past it, Get returns `NotFound` - `TestRedisMemoryProvider_NilClient_ReturnsSentinel` — constructor with empty URL → Get/Set/Delete all return `ErrRedisClientNotInitialized` - `TestMemcachedMemoryProvider_*` — mirror shape - `TestMemcachedMemoryProvider_List_NotSupported` — asserts `ErrListNotSupported`

Build tag: `//go:build integration`. Run via `make test-integration-full` per existing pattern.

Unit tests (CONST-050(A) allows mocks here): use `redis/v9`'s `redismock` package or hand-rolled fake satisfying the minimal interface; assert nil-client sentinel path + method dispatch.

5.2 submodules/storage/ — SKIP-OK integration test

`TestSyncRecording_RealS3Upload` marked `t.Skip("SKIP-OK: #STORAGE-S3-REAL-ROUND37 — requires real S3 bucket; set STORAGE_TEST_BUCKET + AWS creds in env to enable")` per CONST-035 loud-skip taxonomy.

Unit tests: assert nil-client returns `ErrS3UploadNotWired`; assert `RecordingStatusSynced` only set on success path (use mock S3 client satisfying minimal `PutObject` interface).

5.3 submodules/vector_db/ — SKIP-OK integration test

`TestPgvectorClient_Search_Real` marked `t.Skip("SKIP-OK: #VECTORDB-PG-REAL-ROUND37 — requires Postgres+pgvector; set VECTORDB_TEST_DSN to enable")`.

Unit tests: nil-pool returns sentinel; assert SQL composition + pgvector operator (`<=>`) presence via mock interface.

5.4 Paired mutation per CONST-055

For each new sentinel-returning path, the test asserts `errors.Is(err, ErrXxxNotInitialized)` (not just `err != nil`) — meta-test ensures the assertion would fail if the production code returned a different error.

6. Implementation order + commit shape

3 independent submodules → 3 parallel subagents. Per submodule, **one focused fix commit** + tests. Commit messages match the round-31 template (sentinel name in commit body + per-finding rationale).

After all 3 land: meta-repo close-out⁹⁹ commit bumping the 3 submodule pointers + `CONTINUATION.md` narrative + push to all 4 distribution remotes.

CONST-061 merge-first pipeline applies if any submodule's upstream has advanced between fetch and push.

7. Risks + mitigations

Risk	Mitigation
<code>go-redis/v9</code> not in <code>helix_code</code> <code>go.mod</code>	Subagent verifies + runs <code>go mod tidy</code> if needed
<code>bradfitz/gomemcache</code> not in <code>go.mod</code>	Same
AWS SDK v2 not in <code>Storage</code> <code>go.mod</code>	

Risk	Mitigation
	Same; if already on v1, choose path-of-least-resistance per existing pattern
pgxpool not in VectorDB go.mod (round 22 may have left it on lib/pq only)	Same
docker-compose.full-test.yml missing Redis/Memcached	Subagent extends + documents in commit
Pre-existing unrelated test failures in helix_code	Verify via <code>git stash</code> ; document in commit; do NOT block on them
CONST-042 secret-leak risk	All credentials from env vars only; never hardcoded; .env stays gitignored
Cross-submodule divergence during multi-agent push	CONST-061 fetch-before-push + retry on lock-race

8. Out-of-scope (round 38+ candidates)

- CloudFront RSA-SHA1 signing (sibling to S3 upload; round-21 sentinel still in place)
- MCP transport wiring (round 29 candidate)
- Harmony OS distributed SDK (round 28 candidate)
- VisionEngine SSH process management (round 27 + 28 sentinels)
- Connection pooling tuning + failover (perf round)
- TLS cert management UX (security round)

9. Definition of Done

A change is DONE when: 1. Each submodule's real client is wired and Get/Set/Delete (or Upload/Search) delegates to it 2. Sentinels still fire on nil-client config gap 3. Integration tests PASS against real backends (helix_code via docker-compose; Storage + VectorDB via SKIP-OK + env-gated) 4. Unit tests PASS at the provider boundary 5. Paired-mutation meta-test for each sentinel asserts `errors.Is(...)` 6. All 4 distribution remotes converged for each touched submodule + meta-repo 7. CONTINUATION.md updated with close-out narrative

No claim of “done” without pasted terminal output from a real test run (CLAUDE.md §1 Rule 8).